Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №10
з дисципліни "Професійний Python"
на тему: "Production Python: Docker, CI/CD та захист проєкту"

Виконав: Студент групи ФеП-33 [Ваше Прізвище]
Перевірив: Свелеба Сергій Андрійович
Варіант: 7 — Система оцінювання студентів (grade-service)

1. Мета роботи

Ознайомитися з підходами до підготовки Python-проєкту до production-oriented використання. Навчитися оформлювати Python-проєкт як пакет, створювати wheel та sdist, використовувати pyproject.toml, працювати зі змінними середовища, забезпечувати безпеку секретів, створювати Dockerfile, збирати Docker image, запускати контейнер із томами даних, налаштовувати CI/CD у GitHub Actions та готувати проєкт до фінального захисту.

2. Індивідуальний варіант та постановка задачі

Варіант 7 — Система оцінювання студентів (`grade-service` / `grade_service`). У лабораторній роботі №10 проєкт переведено в production-oriented стан на основі FastAPI та SQLite.

Основні завдання відповідно до варіанта:

- Реалізувати REST API для управління студентами та оцінками: ендпоїнти для студентів, дисциплін, оцінок та розрахунку середнього бала (`/students`, `/subjects`, `/grades`, `/students/{id}/average`, обов'язковий `/health`).
- Оформити проєкт як пакет із `pyproject.toml` та зібрати дистрибутиви `wheel` і `sdist`.
- Винести всю конфігурацію в змінні середовища та повністю уникнути збереження секретів у коді чи репозиторії.
- Створити оптимізований Dockerfile, налаштувати volume для персистентності бази даних та healthcheck.
- Налаштувати GitHub Actions для автоматичного виконання лінтингу, перевірки типів, тестів та збірки.

3. Структура проєкту

Проєкт використовує сучасну структуру `src-layout`, яка відокремлює вихідний код від тестового середовища та конфігураційних файлів.

```
grade-service/
├── .github/workflows/ci.yml
├── src/grade_service/
│   ├── __init__.py
│   ├── api.py
│   ├── config.py
│   ├── database.py
│   ├── main.py
│   ├── models.py
│   ├── repositories.py
│   ├── schemas.py
│   └── services.py
├── tests/
│   ├── conftest.py
│   └── test_api.py
├── .dockerignore
├── .env.example
├── .gitignore
├── Dockerfile
├── pyproject.toml
└── README.md
```

4. Опис модулів системи

Модуль / Файл	Призначення
`pyproject.toml`	Метадані пакета, залежності, точка входу `grade-service`, налаштування pytest та ruff.
`config.py`	Клас Settings (pydantic-settings) для читання конфігурації з середовища та `.env`.
`database.py`	Створення SQLAlchemy engine, сесії SessionLocal та ініціалізація бази даних.
`models.py`	ORM-моделі StudentModel та GradeModel зі зв'язками та індексами.
`schemas.py`	Pydantic-схеми валідації вхідних даних та серіалізації відповідей.
`repositories.py`	Шар доступу до даних (StudentRepository, GradeRepository).
`services.py`	Бізнес-логіка (розрахунок середнього бала студента).
`api.py`	Маршрути FastAPI, обробка помилок, ендпоїнт `/health`, логування.

5. Теоретичні відомості

Production-oriented підхід вимагає відтворення середовища без покладання на локальні налаштування розробника. Використання пакетів через `python -m build` гарантує стандартизацію дистрибутивів (`.whl` та `.tar.gz`). Усі секрети виключаються з коду та Git, передаючись через змінні середовища процесу або конфігураційні файли на кшталт `.env`, які ігноруються в `.gitignore`.

6. Інструменти забезпечення якості коду

Інструмент	Команда	Призначення
Ruff (Lint)	`ruff check`	Швидкий пошук синтаксичних та стильових помилок.
Ruff (Format)	`ruff format --check`	Перевірка єдиного стандарту форматування коду.
Mypy	`mypy src`	Статична перевірка типів у всьому коді пакета.
Pytest	`pytest -v`	Автоматизоване виконання модульних та інтеграційних тестів.

7. Реалізація та конфігурація

Управління конфігурацією реалізовано через клас `Settings` на базі бібліотеки `pydantic-settings`. Це дозволяє безшовно читати змінні середовища як для локального запуску через Uvicorn, так і всередині Docker-контейнера. Додатково налаштовано коректне логування подій у стандартний потік виведення (`stdout`) із налаштовуваним рівнем деталізації (`LOG_LEVEL`).

Усі реляційні зв'язки між сукупностями студентів та їх оцінками забезпечують цілісність даних на рівні ORM та бази SQLite.

8. CI/CD та Docker

Конвеєр GitHub Actions у файлі `ci.yml` автоматично виконує повний цикл перевірки при кожному `push` або `pull_request`: встановлення залежностей, лінтинг Ruff, перевірку типів Муру, виконання тестів Pytest, збірку wheel/sdist пакету та збірку Docker-образу. Це гарантує незмінність якості коду перед випуском релізу.

9. Контрольні висновки та захист

Під час виконання лабораторної роботи було успішно досягнуто всіх цілей:

- Проєкт переведено на стандартизований `src-layout` та оформлено як інстальований Python-пакет.
- Налаштовано безпечну конфігурацію через змінні середовища без жорстко закодованих секретів.
- Створено production-ready Dockerfile із використанням не-root користувача та персистентного тому для SQLite.
- Налаштовано зелений конвеєр CI у GitHub Actions, що перевіряє код за всіма критеріями якості.

Висновок: Робота продемонструвала повний цикл перетворення локального скрипта на відтворюваний, протестований та контейнеризований production-ready Python-сервіс.